

扫地机器人 AI 客服 Agent

面试回答速查手册

项目解析 · RAG 问答 Agent 知识点 · 高频追问

基于 LangGraph ReAct + RAG 混合检索 + FastAPI/Streamlit 双入口 + SQLite 持久化

第一部分 项目整体解析与面试回答模版

一、一句话自我介绍（电梯陈述）

我做了一个扫地机器人智能客服 Agent，基于 LangGraph 的 ReAct 架构把大模型、工具调用和 RAG 串起来：用户问“怎么保养”，Agent 判断需不需要查资料，需要就调 RAG 工具，走“查询改写 → 稠密向量 + BM25 稀疏检索 → RRF 融合 → LLM 总结”的链路回答；用户要“生成使用报告”，Agent 按固定流程调工具拿用户 ID、月份、外部记录，并通过中间件动态切换报告提示词生成 Markdown 报告；还支持联网天气、多轮记忆、会话管理。整体做了 FastAPI + Streamlit 双入口、SQLite 持久化、Docker 部署，并搭了检索/生成/延迟的量化评测体系。

二、整体架构

分层 + 单一职责，目录如下：

- ├── app.py Streamlit 前端（聊天 UI + 会话侧边栏）
- ├── api/server.py FastAPI 后端（HTTP/SSE + 鉴权 + 限流）
- ├── agent/ ReAct Agent + tools + middleware
- ├── rag/ 向量库 / 混合检索 / 查询改写 / 总结服务
- ├── model/factory.py 模型工厂（LLM + Embedding）
- ├── prompts/ 提示词（主/总结/改写/报告）
- ├── config/*.yaml rag/chroma/prompts/agent 配置
- ├── data/ 知识库语料 + external/records.csv + conversations.db
- ├── utils/ DB / 配置 / 环境 / 日志 / 文件 / 路径
- ├── eval/ 评测体系（检索/生成/延迟 + 黄金集）
- └── test/ pytest 测试

一条问答的数据流：用户 → app.py/api → ReactAgent.execute_stream(session_id) → LangGraph Agent（ReAct 循环）→ 调工具 → 流式返回 → 写 SQLite。

三、为什么用 LangGraph / ReAct

ReAct = 思考(Thought) + 行动(Action) + 观察(Observation) 循环。用 langchain.agents.create_agent 绑定 model、system_prompt、tools、middleware、checkpoint。系统提示词强制“思考→行动→观察→再思考”，信息不足就调工具，5 次工具调用后仍不足就回“我不知道”。相比纯 prompt，模型能主动决定“要不要查、查什么、查完再想”，可追踪决策、减少幻觉。中间件做横切能力，不侵入业务。

四、中间件 Middleware

- monitor_tool (@wrap_tool_call)：记录工具名/入参/成败；调用 fill_context_for_report 时把 runtime.context["report"]=True 写进上下文。
- log_before_model (@before_model)：模型调用前打日志。
- report_prompt_switch (@dynamic_prompt)：按 context["report"] 动态切“客服提示词 / 报告写手提示词”，同一 Agent 两种模式。

亮点：把“报告场景”从“模型硬背流程”升级为“工具 + 中间件状态机”，更可控可测。

五、多轮记忆 / 会话隔离

用 SqliteSaver 做持久化 checkpoint，config={"configurable":{"thread_id": session_id}}

决定会话归属——同一 thread_id 自动带历史，不同 thread_id 隔离。会话 uuid 与 thread_id 共用同一个 uuid，UI 和 Agent 记忆天然对齐；删除会话时同步 delete_thread。checkpoint 落 SQLite，重启后仍能按

thread_id 找回，属于持久化记忆。

六、RAG 完整链路

离线入库：TextLoader/PyPDFLoader 读 txt/pdf → RecursiveCharacterTextSplitter (200 字 + 20 重叠，中英文标点分隔) → DashScopeEmbeddings(text-embedding-v4) 向量化 → 写 ChromaDB 持久化；文件算 MD5 做增量入库，跳过已存在文件。

在线检索：查询改写 → 双路召回（稠密 + BM25）→ RRF 融合。

生成：chunk 拼成 [1]: 内容 | metadata 放进 prompt，LLM 基于资料总结，prompt 约束“不编造、只基于资料”；用 LCEL PromptTemplate | model | StrOutputParser()。

七、混合检索 + RRF

稠密路抓语义 (Chroma.similarity_search)，稀疏路抓精确关键词 (BM25Okapi + jieba 分词)，互补。RRF: $\text{score}(d) = \sum$

$1/(k+\text{rank})$ ，不依赖两路分数量纲、只按排名融合，天然去重公平。chunk 正文 md5 作唯一 key 对齐去重。BM25 索引做版本化懒构建（语料 count + id 指纹判断是否重建）+ 线程锁 + 空语料不建索引防除零。

八、查询改写

解决口语化、指代、召回不足（如“我家 50 平买哪个好”）。QueryRewriter 用 LLM 生成“改写问题 + 多个子查询”多路召回。_parse 做容错（整体 JSON → 正则提取 [...] → 按行兜底），异常或开关关闭回退原查询。

九、工具设计

- rag_summarize: RAG 检索总结。
- get_weather: 联网 Open-Meteo 实时天气 + 未来三天。
- get_user_location / get_user_id / get_current_month: 用户上下文 (mock)。
- fetch_external_data: 读 records.csv 用户月度记录。
- fill_context_for_report: 报告场景前置触发工具，触发中间件切提示词。

报告强约束：主提示词硬性要求“get_user_id → get_current_month → fill_context_for_report → fetch_external_data”固定流程，未调前置工具禁止报告操作——业务规则工程化。

十、模型工厂

BaseModelFactory(ABC) 定义 generator(), ChatModelFactory 生成

ChatTongyi(qwen3-max), EmbeddingsFactory 生成

DashScopeEmbeddings(text-embedding-v4)，模块加载时生成全局单例，业务只 import 单例不关心实现。Key 从 .env 读入并 os.environ.setdefault 注入。

十一、数据持久化

SQLite 两张表 conversations / messages，外键级联删除；WAL 让读写不互斥（适合 Streamlit 和 FastAPI 双进程共享 db），busy_timeout 防写锁；懒创建会话（首次提问才建库），第一句问题截前 20 字做标题；get_conn() 单例 + check_same_thread=False。

十二、API 设计

FastAPI: /health (含 SELECT 1 检查 db)、/chat (JSON 或 SSE 流式)、/conversations (列表/删除)。SSE 用 StreamingResponse + iterate_in_threadpool 放线程池, finally 里落库防流中断丢记录; 鉴权用 hmac.compare_digest 防时序攻击; 限流用内存滑动窗口 (每 IP 30 次/分钟); 请求/响应用 Pydantic 校验。

十三、配置与工程化

YAML 管业务参数、.env 管密钥、Settings 集中管理; 日志双 handler (控制台 INFO + 文件 DEBUG, 按天切分); get_abs_path 基于 __file__ 定位根目录不依赖启动目录; Docker 多阶段构建 + 国内镜像源, docker-compose 起 agent-api + agent-ui 两服务, 卷持久化 chroma_db/data/logs, 带 healthcheck。

十四、评测体系

分检索、生成、延迟三段量化。

- 检索: hit@k、recall@k、mrr@k、ndcg@k、context_precision@k、recall_full; 四基线对比 (纯稠密 / 纯 BM25 / 混合 RRF / 混合+改写)。
- 生成: LLM-as-Judge (独立 qwen-plus 裁判), 评 faithfulness (忠实度/幻觉)、relevance (相关性)、completeness (完整度)。
- 延迟/成本: 分阶段计时 (改写→稠密→稀疏融合→生成), 统计 P50/P95、token、成本。
- 黄金集: 从语料自动解析 QA/故障/知识条目, 答案“最长前缀匹配”映射相关 chunk (md5 稳定键), 并生成同义/口语化变体检验鲁棒性, validate_gold 做覆盖率/重复率校验。

十五、可主动讲的加分点

- 完整闭环: Agent→RAG→工具→记忆→持久化→API→前端→评测→部署。
- 工程细节扎实: MD5 增量入库、BM25 版本化懒构建+锁、查询改写容错、RRF 去重、SSE 断流落库、外键级联+WAL。
- 可观测可评测: middleware 日志 + 三层评测 (检索/生成/延迟)。
- 业务场景真实: 报告强约束流程、天气适配、个性化报告。

十六、高频追问

- RRF 和加权平均区别: 两路分数量纲不同不能直接加, RRF 只按排名、更鲁棒且天然去重。
- 为什么 BM25 而不纯向量: 中文型号/专有名词向量易漏, BM25 关键词命中更稳, 混合互补。
- chunk 怎么定: 200 字 + 20 重叠, 平衡语义完整与粒度, 分隔符覆盖中英文标点。
- 怎么防幻觉: RAG prompt 强约束“只基于资料” + faithfulness 评测 + Agent 信息不足回“我不知道”。
- 多进程部署问题: agent 单例按进程复制, 限流/缓存是进程内内存态, 多实例需换 Redis 等共享存储。

第二部分 RAG 问答 Agent 知识点清单

一、RAG 基础概念与动机

- RAG 是什么：Retrieval-Augmented Generation，检索增强生成。先检索外部知识，再把结果作上下文交给 LLM 生成，解决知识过时、幻觉、私有知识不懂的问题。
- 为什么 RAG 而非微调：成本低、知识可即时更新（只换语料不重训）、可溯源、可控性更强。
- RAG 三段式：离线索引（Indexing）→ 在线检索（Retrieval）→ 生成（Generation）。
- RAG 与 Agent 的关系：本项目里 RAG 封装成 rag_summarize 工具，由 ReAct Agent 按需调用——“Agent 决定要不要查、查什么”。

二、离线入库（Indexing / 文档处理）

- 文档加载：多格式读取，TextLoader (txt)、PyPDFLoader (pdf)，本项目限定 txt/pdf 白名单。
- 文本分块：RecursiveCharacterTextSplitter，chunk_size=200、chunk_overlap=20。overlap 防语义切断；递归分隔符按 \n\n → \n → 中英文标点 优先级逐级切。
- 向量化：DashScopeEmbeddings(text-embedding-v4)，语义相近文本向量距离近。
- 向量库入库：写 ChromaDB，persist_directory 持久化到磁盘。
- 增量更新：文件算 MD5 指纹，已入库文件跳过，只灌新增/变更文件。
- chunk 与 metadata：每个 chunk 带来源等 metadata，生成时拼进 prompt 可溯源。

三、在线检索（Retrieval）

- 稠密检索（Dense）：query 向量化后 similarity_search 找语义最像的 k 条，擅长同义改写、口语化。
- 稀疏检索（Sparse）：BM25 关键词匹配，rank_bm25.BM25Okapi，中文用 jieba 分词，擅长精确词、型号、专有名词。
- 为什么混合检索：向量会漏精确关键词，BM25 会漏语义相近但用词不同的内容，互补。
- RRF 融合： $\text{score}(d) = \sum 1/(k + \text{rank}(d))$ ，不依赖分数量纲、只按排名融合，天然去重公平。
- 文档去重对齐：chunk 正文 md5 作唯一 key，稠密/稀疏两路拿到同一文档时合并。
- 检索返回 top_n：稠密 dense_top_n=10、稀疏 sparse_top_n=10、k 控制最终上下文大小。

四、查询优化（Query Processing）

- 查询改写：LLM 把口语化、有指代的问题改写成更完整、更适合检索的问题。
- 子查询拆分：从不同侧面拆出多个子查询，多路召回提升 recall。
- 多查询融合：多个改写查询分别检索后统一 RRF 融合。
- 容错解析：LLM 输出 JSON 不干净时，整体解析 → 正则提取 [...] → 按行兜底。
- 降级策略：改写异常或开关关闭时回退原查询，保证链路可用。

五、生成（Generation）

- 上下文拼接：检索到的 chunk 拼成 [1]: 内容 | metadata 编号格式喂给 LLM。
- RAG 总结提示词：约束“只基于参考资料、不编造、不主观推断、聚焦问题”，本质是防幻觉。
- LCEL 链式写法：PromptTemplate | model | StrOutputParser()。
- RAG summarize 服务：把“改写→检索→拼接→生成”封装成完整服务（RagSummarizeService）。

六、向量库 (Vector Store)

- ChromaDB

基本概念: collection (集合)、document (文档)、embedding (向量)、metadata (元数据)。

- 持久化 vs 内存: persist_directory 落盘, 服务重启不丢。
- as_retriever / similarity_search: 标准检索器接口与相似度搜索。
- 轻量语料指纹: get(include=[]) 只拉 id 不拉全文, 算 md5 判断语料是否变化, 用于 BM25 版本化重建。

七、RAG Agent 的结合点

- RAG 作为工具: rag_summarize(query) 用 @tool 装饰, Agent 自主决策调用。
- ReAct 循环 + RAG: 模型判断“常识够不够, 不够就调 RAG 工具”, 工具返回后再判断够不够。
- 报告场景动态提示词: 中间件 dynamic_prompt 在“普通 RAG 问答”和“报告生成”之间切换。
- 单例复用: get_rag_service() 进程内单例, 避免每次工具调用重建 Chroma 客户端 / Prompt / BM25 索引。

八、评测 (Evaluation, RAG 专属)

- 检索指标: hit@k、recall@k、mrr@k、ndcg@k、context_precision@k (相关 chunk 是否排前、无关是否稀释上下文)、recall_full (整批召回率)。
- 四基线对比: 纯稠密 / 纯 BM25 / 混合 RRF / 混合+改写, 量化每项改造收益。
- 生成指标 (LLM-as-Judge): faithfulness (忠实度/是否幻觉)、relevance (相关性)、completeness (完整度)。
- 黄金集: 答案通过“最长前缀匹配”映射到相关 chunk (md5 稳定键), 自动派生可复现。
- 同义变体测试: 生成口语化/同义 query 变体, 检验“换个说法还能不能召回”。
- 延迟/成本评测: 分阶段计时 (改写→稠密→稀疏融合→生成), 统计 P50/P95、token、成本。

九、工程优化与踩坑点

- BM25 版本化懒构建: 语料“count + id 指纹”变化才重建, 进程内只建一次, 加线程锁防并发。
- 空语料保护: 空库时不建 BM25 索引, 避免除零。
- 分词兜底: jieba 缺失时退化为“CJK 二元组 + 英文单词”, 零依赖可用。
- 增量入库去重: MD5 防重复灌入。
- 性能权衡: top_n、chunk_size、overlap 是“召回 vs 上下文长度 vs 成本”的平衡点。